Apellidos y Nombre: .....Grupo:.....DNI: .....

Examen Parcial 2 (Temas 8, 9, 10, 11, 12, 13, 14)

- Duración de esta parte: 2.30 horas. Escribid las respuestas en el enunciado y entregad todas sus hojas con apellidos y nombre en la cabecera.
- No podéis utilizar calculadora, móvil, apuntes, etc. En hojas 5-6 se os da una "chuleta" con información útil para realizar los ejercicios.
- La solución se publicará la tarde del 8 Enero 2024 y las notas el 9 Enero 2024.

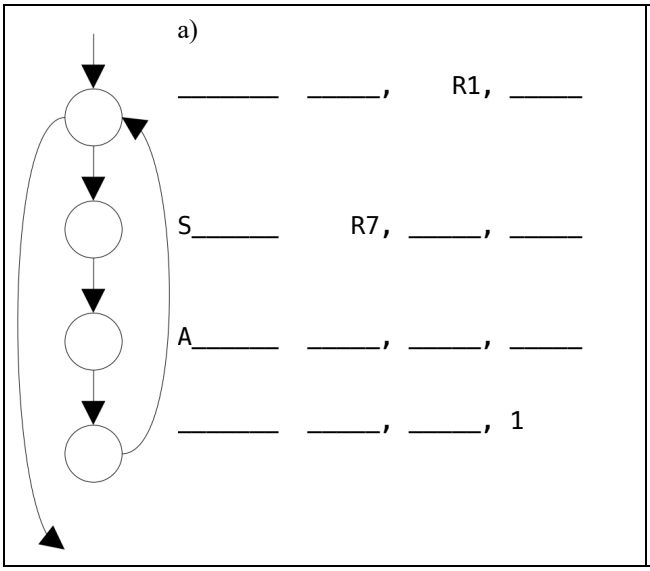
Pregunta 1 (2 puntos)

Dado el siguiente pseudo-código

```
while ( R0>R1 ) {
       R2 = R2*3;
       R0 = R0 -1;
}
```

y considerando que los registros `R0, R1,` y `R2` contienen números naturales, completad:

a) El grafo de estados de la UC de **propósito específico** para que, junto con la UPG, formen un PPE que resuelva el problema propuesto. No faltan nodos, faltan arcos y sus etiquetas (`z`, `!z`, o `x`) y el mnemotécnico en cada nodo. `R7` es un registro temporal.
b) El código en ensamblador SISA para realizarla en el sistema formado por la UC **general** junto con la UPG y el subsistema de entrada/salida (UCG+UPG+$IO_{key\text{-}print}$). Se utilizan los registros `R6` y `R7` para almacenar valores temporales.



Pregunta 2 (2,25 puntos)

a) Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de una instrucción en el SISC Von Neumann. Escribid el valor de los bits de la palabra de control que genera el bloque SISC CONTROL UNIT durante el ciclo a que hace referencia cada apartado. Poned `x` siempre que no se pueda saber el valor de un bit (ya que no sabemos cómo se han implementado las `x` en la ROM_OUT). Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el `IR` en ese ciclo. Podéis ver el grafo de estados de Moore de la UC en el anexo. Suponed que el contenido de todos los registros, `Rk` para `k=0, . . . , 7`, antes de ejecutarse cada instrucción es 0.

Apartado	Nodo (salida)	Instrucción en IR (ensamblador)	@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	LdIr	LdPc	Byte	Alu/R@	R@/Pc	N (Hexa)	ADDR-IO (hexa)
A	Movi	MOVI R4, 0x88																			
B	Ldb	LDB R6, 0x32(R7)																			
C	D	ADD R1, R2, R3																			

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado).

Nodo/Estado (Mnemo_salida)	Instrucción en IR (en ensamblador)	Contenido ROM_OUT (en hexadecimal)
Bnz	BNZ R2, -4	0x
D	JALR R6, R4	0x
Ldb	LDB R2, 2(R1)	0x

Pregunta 3 (2,25 puntos)

El programa ensablador se ha traducido a lenguaje máquina para ser ejecutado en el SISC Von Neumann. La sección .data se situa a partir de la dirección 0x80A0 de memoria y a continuación la sección .text. Una vez ensamblado y cargado el programa en memoria:

```
.data
     N = 5
A:   .space 1
B:   .byte -27
     .even
C:   .word 2, -5, 264, -63, 23
D:   .byte 58, -64, 32, 0, -7
     .even
E:   .space 20

.text
L1: IN R0, KEY-STATUS
    BZ R0, L1
    IN R0, KEY-DATA
    MOVI R2, LO(C)
    MOVHI R2, HI(C)
    MOVI R1, N
    MOVI R4, 1
L2: LD R5, 0(R2)
    ...
```

a) ¿A qué direcciones de memoria corresponden las etiquetas, o direcciones simbólicas siguientes?

C = 0x
D = 0x
L2 = 0x

b) ¿Cuál es la dirección de memoria y su contenido donde se han almacenado las siguientes instrucciones?

MOVHI	R2, HI(C)	→ Memw[0x	] = 0x
BZ	R0, L1	→ Memw[0x	] = 0x
LD	R5, 0(R2)	→ Memw[0x	] = 0x

Pregunta 4 (3,5 puntos)

El diseñador del lenguaje máquina SISA considera que es preciso añadir una nueva instrucción al repertorio de instrucciones. Se trata de una instrucción de salto indexado (indexed jump, JI) que permite saltar a la dirección de código indicada por la posición n-ésima de un vector almacenado en memoria, donde *n* es un natural menor que 128. Como la instrucción JALR, esta nueva instrucción también devuelve la dirección de código correspondiente al secuenciamiento implícito.

Sintaxis: JI Ra, N7
Codificación: 1111 aaa x 0nnn nnnn
Semántica: tmp = PC + 2; PC = Memw[Ra + 2 * N7]; Ra = tmp;

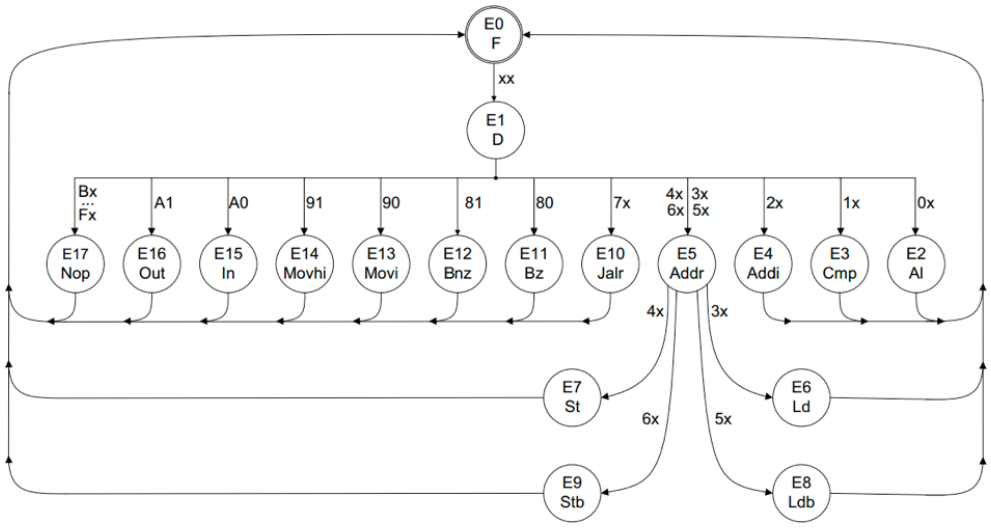
La codificación de la instrucción exige que el bit 7 valga '0'; en cambio, el valor del bit 8 es indiferente. Tened en cuenta que Ra es tanto registro fuente como registro destino. Por ejemplo, si R3=0xBECA, PC=0xAD10 y el vector de *words* almacenado a partir de la dirección 0xBECA contiene los words 0xABBA, 0xACDC, 0xCAFE, 0xBEBE, 0xBABE, 0xFACE, ... en este orden, la ejecución de JI R3, 0 provocaría que PC=0xABBA y R3=0xAD12; en cambio, la ejecución de JI R3, 5 provocaría que PC=0xFACE y R3=0xAD12.

a) Consideramos un programa que ejecuta 2.000 instrucciones rápidas y 1.000 de acceso a memoria. Reescribimos el programa utilizando la instrucción JI y obtenemos una nueva versión del programa que ejecuta 1.600 instrucciones rápidas, 900 instrucciones de acceso a memoria y 200 instrucciones JI. Suponiendo que la implementación de la nueva instrucción tarde 6 ciclos (incluyendo *Fetch* y *Decode*) y que no impacte en el Tc, indicad cuántos ciclos tarda la ejecución de cada versión del programa y qué porcentaje de reducción en el tiempo de ejecución observaríamos respecto al tiempo de ejecución del programa original. (Indicad el cálculo en función del número y tipo de instrucciones, así como el resultado final)

Apellidos y Nombre: .....Grupo:.....DNI: .....

b) Sin modificar el hardware y sólo modificando el contenido de las ROM's, completad el diseño del computador para que ejecute, además de las instrucciones originales, la instrucción JI en 6 ciclos (incluyendo F y D).

b1) Indicad qué modificaciones introduciríais en el grafo de estados de la unidad de control



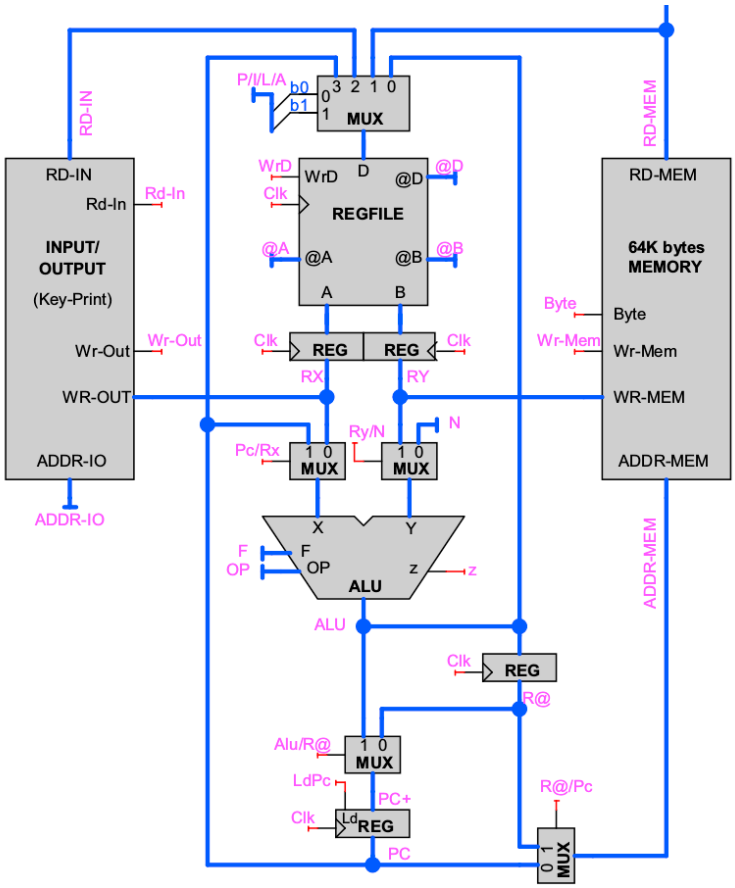
b2) Indicad el contenido de las filas de la ROM_OUT que sea preciso modificar así como las acciones a realizar (la tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean necesarias todas)

@ROM	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0

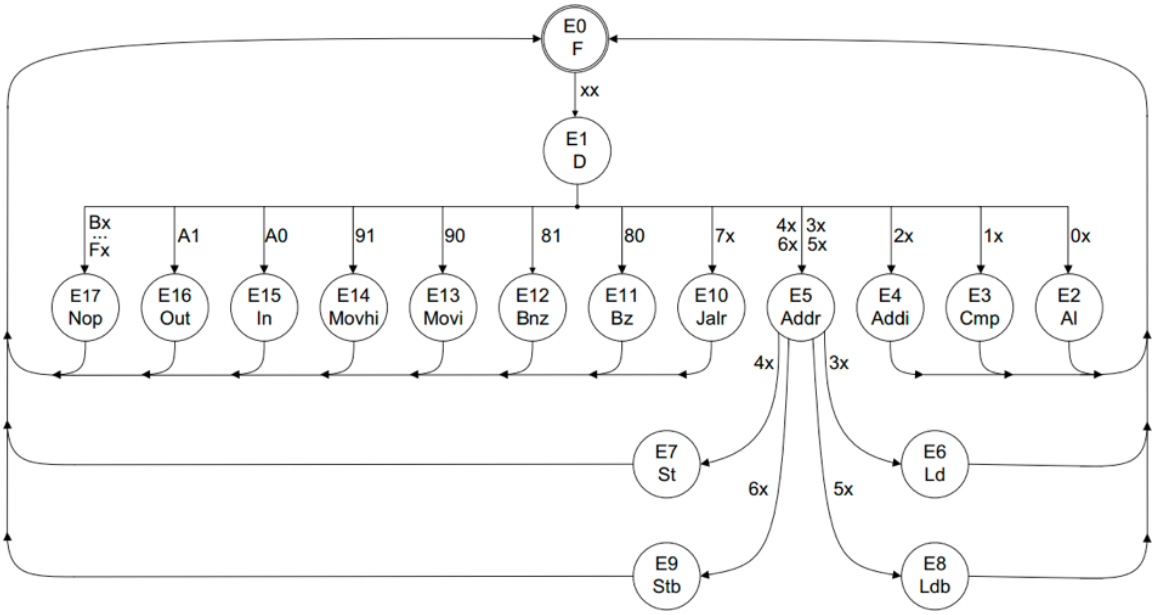
Acciones asociadas al estado (en lenguaje de transferencia de registros)

c) Si pudiéramos modificar el hardware de la Unidad de Proceso y el de la Unidad de Control),

c1) ¿Cómo modificaríais el hardware de la UPG para reducir el tiempo de ejecución de la instrucción JI a 4 ciclos (incluyendo *Fetch* y *Decode*)? Podéis modificar buses y añadir multiplexor(es), buses y señales de control, pero no podéis modificar ni los bloques ni el uso de las señales de control existentes. Para poder mantener el tiempo de ciclo, no es válido calcular la dirección de memoria y acceder a memoria en el mismo ciclo.



c2) ¿Cómo modificaríais consecuentemente el grafo de estados de la UC?



c3) Indicad el contenido total de las filas de la ROM_OUT que sea preciso añadir, así como las acciones a realizar; disponéis de espacio para indicar dos nuevas señales de control (es posible que no sean necesarias todas). Indicad también el valor de la(s) nueva(s) señal(es) en al menos dos de las filas de la ROM_OUT ya existentes en las que la(s) nueva(s) señal(es) de control no tenga(n) el valor x. La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean necesarias todas.

Apellidos y Nombre: .....Grupo:.....DNI: .....

@ROM	New1	New2	Bnz	Bz	WrMem	RdIn	WrOut	WrD	Ldlr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0

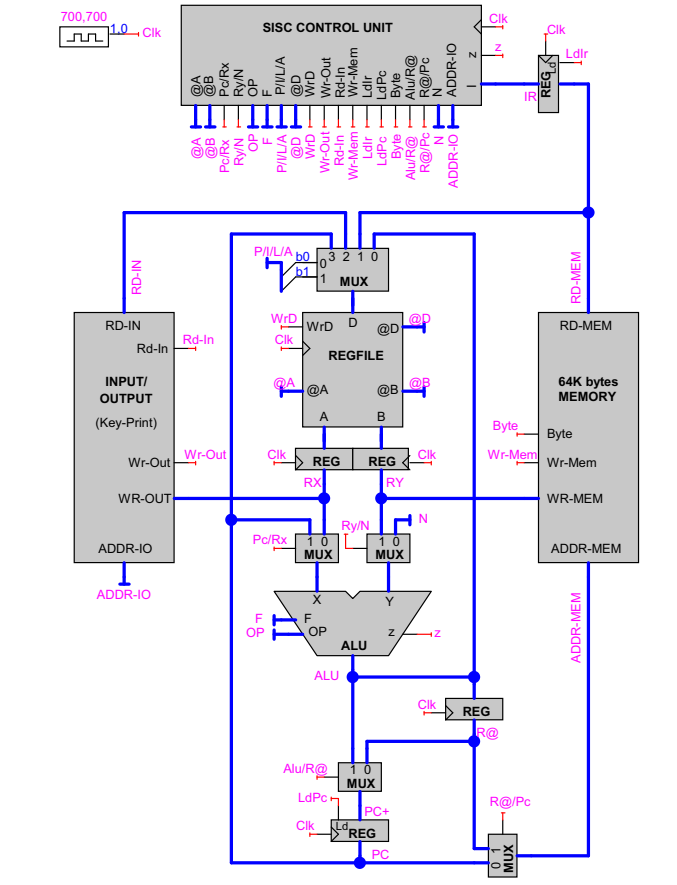
Acciones asociadas al estado (en lenguaje de transferencia de registros)

c4) Actualizad el cálculo del apartado a) considerando que la instrucción JI tarda 4 ciclos.

Material Auxiliar

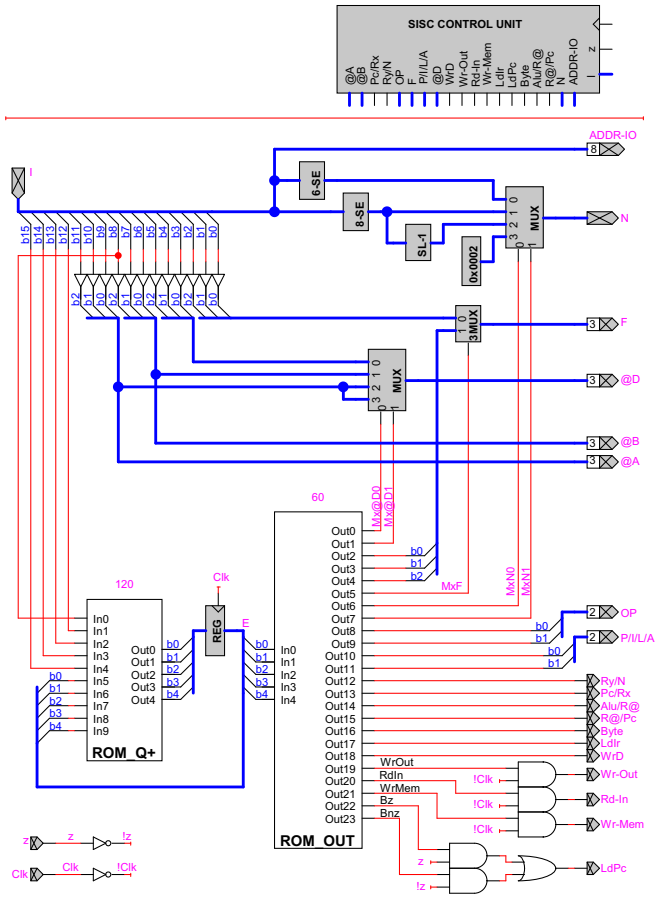
SISC

Simple Instrucction Set Computer **SISC**



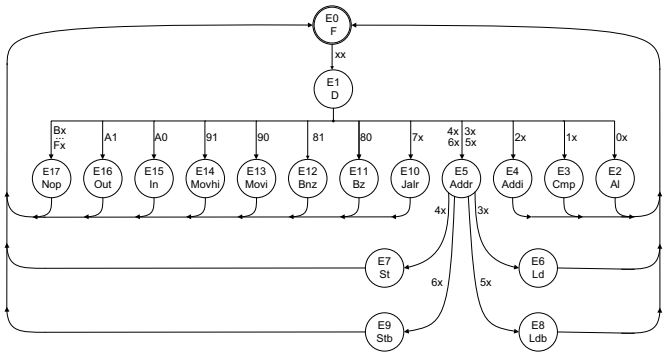
Bloque SISC CONTROL UNIT

Control Unit of the SISC Computer **SISC CONTROL UNIT**



SISA. Formato y codificación

16-bit Instruction																Mnemonic	Format
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	0	0	0	a	a	a	b	b	b	d	d	d	f	f	f	AND, OR, XOR, NOT, ADD, SUB, SHA, SHL	3R
0	0	0	1	a	a	a	b	b	b	d	d	d	f	f	f	CMPLT, CMPLE, -,CMPEQ, CMPLTU,CMPLEU, -, -	
0	0	1	0	a	a	a	d	d	d	n	n	n	n	n	n	ADDI	2R
0	0	1	1	a	a	a	d	d	d	n	n	n	n	n	n	LD	
0	1	0	0	a	a	a	b	b	b	n	n	n	n	n	n	ST	
0	1	0	1	a	a	a	d	d	d	n	n	n	n	n	n	LDB	
0	1	1	0	a	a	a	b	b	b	n	n	n	n	n	n	STB	
0	1	1	1	a	a	a	d	d	d	x	x	x	x	x	x	JALR	
1	0	0	0	a	a	a	0	n	n	n	n	n	n	n	n	BZ	1R
							1									BNZ	
1	0	0	1	d	d	d	0	n	n	n	n	n	n	n	n	MOVI	
							1									MOVHI	
1	0	1	0	d	d	d	0	n	n	n	n	n	n	n	n	IN	
				a	a	a	1									OUT	



Funcionalidad de la ALU

F			OP			
b_2	b_1	b_0	11	10	01	00
0	0	0	---	X	CMPLT (X, Y)	AND (X, Y)
0	0	1	---	Y	CMPLE (X, Y)	OR (X, Y)
0	1	0	---	MOVHI(X, Y)	---	XOR(X, Y)
0	1	1	---	X&(~1)	CMPEQ (X, Y)	NOT (X)
1	0	0	---	---	CMPLTU (X, Y)	ADD (X, Y)
1	0	1	---	---	CMPLEU (X, Y)	SUB (X, Y)
1	1	0	---	---	---	SHA(X, Y)
1	1	1	---	---	---	SHL(X, Y)